

Pianificazione Appelli

Discorso di presentazione

Gabriele Fiorucci – 740746

Nota: ogni sezione corrisponde a una slide. Il testo è pensato per essere letto o parafrasato ad alta voce; le indicazioni tra parentesi quadre sono note di regia e non vanno pronunciate.

Apertura [slide titolo]

Buongiorno. Mi presento: sono Gabriele Fiorucci, matricola 740746, e oggi vi illustro il progetto che ho sviluppato per il corso di Progettazione di Applicazioni Web Full Stack.

L'applicazione si chiama **Pianificazione Appelli** ed è un sistema pensato per un contesto universitario: il suo scopo è permettere alla segreteria e ai docenti di organizzare in modo ordinato gli appelli d'esame, rispettando una serie di regole e vincoli che nella realtà spesso vengono gestiti a mano. Vi mostrerò sia come è fatta “dietro le quinte”, cioè l'architettura, sia come si presenta all'utente.

Scaletta

Prima di entrare nel dettaglio, vi anticipo il percorso che seguiremo.

Partiamo dalla **progettazione dei dati**: il diagramma Entità-Relazioni, il diagramma degli utenti e la matrice CRUD, cioè chi può fare cosa. Poi passiamo al **back-end**, dove vi mostro come è organizzato il codice del server. Vedremo quindi un **esempio completo end-to-end**, cioè come un dato viaggia dal database fino alla pagina che l'utente vede. Da lì passiamo al **front-end**, con le schermate vere e proprie. Infine chiudiamo con la parte di **progettazione dell'ipertesto**: il progetto Coarse e lo schema di navigazione, che è il metodo di design che abbiamo seguito durante il corso.

Diagramma ER

Questo è il diagramma Entità-Relazioni, il cuore del modello dei dati.

Al vertice c'è l'entità **Utente**, con i campi fondamentali: identificativo, nome, email, password e ruolo. Il ruolo può essere *Docente* oppure *Segreteria*. Da Utente parte una relazione di tipo **IS-A**, cioè di specializzazione: un utente è un docente *oppure* è un membro della segreteria. Questo mi permette di tenere i dati comuni in un unico posto e di aggiungere solo le informazioni specifiche dove servono.

Scendendo nel diagramma troviamo le entità di dominio. Il **Docente** *insegna* degli **Insegnamenti** (le materie), e ogni insegnamento *appartiene* a un **Corso di Laurea**. Sempre il docente *pianifica* gli **Appelli**: ogni appello *riguarda* un insegnamento ed è *collocato* all'interno di una **Sessione d'esame**.

Vorrei farvi notare alcuni attributi importanti. L'appello ha data, ora di inizio e ora di fine, il *tipo di aula* – standard, laboratorio informatico, di elettronica o di sensori – e il *tipo di esame*, orale o scritto. La sessione, invece, ha due intervalli distinti: il periodo in cui si svolge la sessione e la *finestra di pianificazione*, cioè il periodo in cui i docenti possono effettivamente inserire gli appelli. Questa distinzione tra “quando si fa l'esame” e “quando si può programmarlo” è un punto centrale della logica dell'applicazione.

Diagramma degli utenti

Qui vediamo la gerarchia degli utenti dal punto di vista dei ruoli.

In cima c'è l'**utente generico**, che si divide subito in due categorie: *registrato* e *non registrato*. L'utente non registrato è chi non ha ancora fatto il login: in pratica può solo vedere la pagina di accesso. L'utente registrato, una volta autenticato, si specializza ulteriormente in **Docente** o **Segreteria**.

Questa struttura ad albero è coerente con il diagramma ER di prima e serve a guidare tutte le decisioni sui permessi: ogni livello dell'albero corrisponde a ciò che quel tipo di utente può vedere e fare nell'applicazione.

Matrice CRUD

Questa tabella riassume in modo compatto i permessi: per ogni risorsa indico, per ciascun ruolo, quali delle quattro operazioni – *Create, Read, Update, Delete* – sono consentite.

La logica di fondo è semplice: la **Segreteria fa da amministratore** e ha pieno controllo su quasi tutte le risorse – corsi di laurea, insegnamenti, sessioni, docenti. Il **Docente**, invece, ha permessi più ristretti e mirati ai *propri* dati.

Vi evidenzio i casi più interessanti, quelli “tra parentesi”. Sugli **Appelli** il docente ha pieno controllo, ma solo sui propri: può crearli, modificarli ed eliminarli, mentre legge tutti gli altri. Sulle proprie informazioni di profilo, sia docente sia segreteria possono leggere e aggiornare *sé stessi*, ma non gli altri: è la regola del *self-only*. Sulle **materie** e sui **corsi** il docente ha sola lettura, perché quelle le gestisce la segreteria. L'utente non registrato, come vedete, non ha alcun permesso: la colonna è vuota.

Questa matrice non è solo documentazione: è esattamente ciò che ho implementato lato server con le guardie sui ruoli.

Back-End – struttura generale

Passiamo al codice. Il progetto è un monorepo: front-end e back-end convivono nello stesso workspace, organizzato con Nx.

Il back-end è costruito con **NestJS** e si trova nella cartella `libs/server`, divisa in sottocartelle specializzate, ciascuna con una responsabilità chiara. C'è **auth** per l'autenticazione, cioè il login e la generazione del token; **security** per la gestione dei permessi dopo l'autenticazione, cioè le guardie sui ruoli; **users** per la gestione degli utenti. Infine **exam-planning**, che contiene la logica vera e propria del dominio: corsi, materie, sessioni, appelli, docenti e segretari.

Separata da queste c'è la cartella **database**, che si occupa solo della connessione a PostgreSQL tramite il file **database.module.ts**. Tenere la connessione isolata in un unico punto è una scelta voluta: la configurazione sta tutta lì.

Authentication, Security & Users

Apro brevemente le tre cartelle “di supporto”.

In **security** ho i mattoni condivisi: le **guardie**, cioè **jwt-auth.guard** che verifica il token e **roles.guard** che verifica il ruolo; i **decoratori** come **@CurrentUser** e **@Roles**, che mi permettono di scrivere i controlli in modo dichiarativo e leggibile; e l'*enum* dei ruoli.

In **auth** c'è tutta la logica di accesso: il controller con l'endpoint di login, le **strategie** di Passport – una “local” che valida email e password, e una “jwt” che valida il token sulle richieste successive – e le interfacce che descrivono la forma del token e della risposta.

In **users** c'è l'entità Utente, il suo repository e il servizio. Anche qui ho mantenuto la separazione tra DTO, entità e logica.

Database Module

Mi soffermo un attimo sul modulo del database, perché è dove si concentra tutta la configurazione.

La connessione avviene tramite **TypeOrmModule**: un **unico modulo centralizzato**. Tutti i parametri sensibili – host, porta, username, password, nome del database – non sono scritti nel codice, ma vengono **letti dal file .env** tramite una funzione **requireEnv**, che ha anche il vantaggio di lanciare un errore chiaro se una variabile manca. Questo garantisce che la configurazione sia in un solo posto e non duplicata.

Due opzioni che vorrei sottolineare: **autoLoadEntities** a **true** fa sì che le entità vengano caricate automaticamente, senza doverle elencare manualmente qui; e **synchronize** a **true** sincronizza lo schema del database con le entità all'avvio. Quest'ultima è comodissima in sviluppo – è quella che uso io – ma va disattivata in produzione, perché potrebbe alterare lo schema in modo automatico.

Logica Back-End

Entriamo nel cuore del progetto, la cartella **exam-planning**.

Per **ogni entità** del dominio ho creato una cartella che segue *sempre lo stesso schema*: ci sono il DTO, le interfacce, l'entity, il repository, il controller e il service. Questa uniformità è una scelta precisa: una volta capito come funziona una entità, le capite tutte, e il codice è facile da estendere.

Oltre alle entità ci sono due **helper** condivisi. Uno per la *gestione centralizzata degli errori*, che traduce gli errori del database in risposte HTTP sensate. L'altro per la *normalizzazione dei nomi*: serve a evitare nomi duplicati anche nei casi insidiosi, cioè nomi che differiscono solo per spazi o per maiuscole e minuscole. In questo modo, ad esempio, due corsi chiamati uno con la maiuscola e uno con la minuscola non vengono trattati come diversi.

Esempio End-to-End – Entity e DTO

Adesso vi mostro concretamente come è fatta un'entità, prendendo come esempio l'Appello. Affianco due file: l'**Entity** a sinistra e il **DTO** a destra.

L'**Entity** descrive come il dato è salvato nel database: le colonne – data, ora di inizio, ora di fine, tipo di aula, tipo di esame – e le relazioni con materia, sessione e docente. Notate il vincolo di unicità sulla coppia data–materia, che impedisce duplicati a livello di database.

Il **DTO**, invece, descrive come il dato arriva dal client e *come va validato*. Per ogni campo ho due tipi di annotazione: quelle di documentazione, che generano automaticamente la pagina Swagger con esempi e descrizioni, e quelle di *validazione*, che controllano i valori in ingresso. Per esempio, l'ora di inizio deve essere un intero tra 0 e 24, la data deve essere in formato valido, il tipo di esame deve essere uno dei valori ammessi. Tutti i messaggi d'errore sono in italiano e pensati per essere chiari all'utente finale.

Il punto chiave è la separazione: l'entità parla con il database, il DTO protegge l'ingresso. Sono due responsabilità diverse, in due file diversi.

Esempio End-to-End – Repository e Service

Saliamo di un livello: **Repository** e **Service**.

Il **Repository** si occupa *solo* di parlare con il database. Contiene CRUD puro: trova tutti, trova per id, trova gli appelli di un docente, e così via. Non contiene logica di business e non lancia eccezioni HTTP: se un dato non esiste, restituisce semplicemente `null`. È volutamente “stupido” e prevedibile.

Il **Service** è dove vive l'intelligenza. Riceve i dati dal repository e applica la **logica di dominio e di autorizzazione**. Qui faccio due cose importanti. La prima: traduco l'assenza di un dato in un errore appropriato – se il repository mi torna `null`, sollevo un `NotFoundException` con un messaggio in italiano. La seconda, fondamentale: con il metodo `toListItem` **trasformo l'entità in una forma “pulita”** prima di restituirla. Questo mi permette di rinominare i campi, di annidare le relazioni come oggetti con id e nome, e soprattutto di **rimuovere i dati sensibili**, come la password dell'utente, che non devono mai uscire dal server.

In pratica: il client non vede mai l'entità grezza del database, ma sempre una versione controllata e sicura.

Esempio End-to-End – Controller e front-end

Ultimo livello del back-end: il **Controller**; e accanto vi mostro già il primo pezzo di front-end.

Il **Controller** espone le API REST. Il suo compito è il routing: associa ogni URL a un metodo del service. Notate le **guardie** sopra ogni rotta – la guardia del token e quella dei ruoli – combinate con il decoratore `@Roles`. È esattamente qui che la matrice CRUD vista prima diventa codice: per esempio l'elenco completo degli appelli è riservato alla segreteria, mentre la rotta “i miei appelli” è riservata al docente e gli restituisce solo i suoi.

Sul **front-end**, la pagina `ExamsPage` si comporta in modo diverso a seconda del ruolo: legge il ruolo dell'utente e decide quale lista chiedere al server. Vi segnalo un dettaglio di esperienza

utente: quando elimino un appello faccio un *aggiornamento ottimistico*, cioè tolgo subito la riga dalla lista senza rifare l'intera chiamata, così l'interfaccia risponde immediatamente.

Esempio End-to-End – CreateExamPage

Questa è la pagina di *creazione* di un appello, che mostra un paio di accorgimenti interessanti.

Primo: i valori iniziali del form possono arrivare **precompilati dall'URL**. Questo serve al collegamento con il calendario: quando il docente clicca su un giorno disponibile, lo porto direttamente qui con materia, data e sessione già impostate.

Secondo: il menu delle sessioni non mostra tutto, ma **filtra solo le sessioni in cui ha ancora senso pianificare**, cioè quelle la cui finestra di pianificazione non è ancora chiusa. In questo modo l'utente non può nemmeno provare a fare qualcosa che il back-end rifiuterebbe: il vincolo è ribadito anche lato interfaccia, per coerenza e per una migliore esperienza.

Descrizione del flusso End-to-End

Riassumo quindi l'intero percorso di un dato, che è il senso di questi ultimi passaggi.

Una richiesta parte dal front-end, entra dal **controller**, scende al **service**, poi al **repository** e infine al **database**. Al ritorno ogni strato ha il suo ruolo: *Entity e DTO* definiscono la struttura dei dati e la validazione; il *Repository* comunica con il database; il *Service* applica la logica di dominio e l'autorizzazione; il *Controller* espone le API REST; e infine nel front-end le pagine in stile “lista” mostrano l'elenco dei dati, mentre le pagine “Create” costruiscono i form per crearli e modificarli.

È un'architettura a strati: ogni livello ha una sola responsabilità, e questo rende il codice testabile e facile da mantenere.

Front-End – panoramica

Passiamo ora a vedere l'applicazione dal punto di vista dell'utente. Il front-end è una *single page application* in React.

Si compone di **tre macro-aree**: Login, Docenti e Segreteria. La pagina di **Login** è comune a tutti e rappresenta il punto d'ingresso per chi non è ancora autenticato. Una volta effettuato l'accesso, l'applicazione apre **automaticamente l'area giusta** in base al ruolo: se sono un docente vedo la sezione docenti, se sono della segreteria vedo la sezione segreteria.

Le due sezioni sono **in gran parte speculari**: la struttura delle pagine è la stessa, cambiano i permessi e alcune funzionalità. Questa simmetria ha reso lo sviluppo più rapido e l'interfaccia più coerente. Vi mostro adesso le schermate.

Login

Questa è la pagina di login. È volutamente essenziale: l'utente inserisce email e password istituzionali e accede.

Dal punto di vista della sicurezza, voglio sottolineare un aspetto: in caso di credenziali errate, il sistema risponde sempre con lo stesso errore generico, sia che l'email non esista, sia che la

password sia sbagliata. Questo per evitare la cosiddetta *user enumeration*, cioè impedire a un malintenzionato di capire quali email sono registrate. Al login andato a buon fine il server restituisce un token JWT, che da quel momento accompagna tutte le richieste.

Dashboard Docenti

Ecco la prima schermata dopo il login di un docente: la **dashboard**, che è anche la home.

In alto trovo tre indicatori di sintesi: quante sessioni sono aperte alla pianificazione, quanti sono i miei appelli e quante materie insegno. Sotto c'è il **calendario**, che è l'elemento centrale di questa pagina.

Il calendario è interattivo: posso scegliere un mio insegnamento dal menu a tendina e vedere immediatamente *in quali giorni posso inserire un appello*. I giorni disponibili vengono evidenziati. Cliccando su un giorno valido vengo portato direttamente al form di creazione già precompilato – è il collegamento di cui parlavo prima. In questo modo la pianificazione diventa visiva e guidata, invece che un form da riempire alla cieca.

Esami (Docente)

Questa è la lista degli appelli del docente. Gli esami sono **raggruppati per sessione**, così il docente ha subito un quadro ordinato di cosa ha programmato e quando.

Per ogni appello vedo data, orario, materia, tipo d'esame e tipo di aula. A destra ho le azioni: *Modifica* ed *Elimina*. Notate però l'ultima riga, l'esame già passato: lì non ci sono pulsanti, c'è la dicitura "Esame concluso". Questo perché, come vedremo nelle regole di dominio, il docente non può toccare un esame già svolto. In alto a destra il pulsante per creare un nuovo esame.

Aggiunta di un nuovo esame

Questo è il form di creazione di un appello dal punto di vista del docente.

Seleziono la materia tra le mie, la sessione tra quelle pianificabili, poi data, ora di inizio, ora di fine, tipo d'esame e tipo di aula. Tutti i campi sono validati, sia lato client per dare un riscontro immediato, sia lato server, che resta l'autorità finale. Il docente, come ho detto, non deve indicare sé stesso come titolare: è il server che forza automaticamente l'appello a essere associato al docente autenticato.

Sessioni (Docente)

Nella pagina delle sessioni il docente vede, in sola lettura, lo stato della programmazione d'ateneo, suddiviso in tre gruppi: **In corso**, cioè le sessioni attualmente aperte; **Pianificate**, cioè quelle future; e **Passate**, già concluse.

Per ogni sessione mostro due intervalli distinti: il periodo della sessione e la finestra di pianificazione degli appelli. È la stessa distinzione che avevamo nel modello ER, e qui diventa evidente all'utente: il docente capisce a colpo d'occhio entro quando deve inserire i propri appelli per una certa sessione.

Insegnamenti (Docente)

Qui il docente consulta i **propri insegnamenti**: nome della materia, anno di corso, CFU, corso di laurea e dipartimento.

È una vista di sola lettura, coerente con la matrice CRUD: gli insegnamenti li crea e li gestisce la segreteria, il docente li vede soltanto. Gli serve per sapere quali materie può poi usare in fase di creazione degli appelli.

Corsi di laurea (Docente)

Analogamente, questa è la lista dei corsi di laurea a cui il docente è collegato attraverso i suoi insegnamenti: nome, durata in anni e dipartimento. Anche questa è sola lettura.

Area personale (Docente)

Ogni utente ha la propria **area personale**. Qui il docente può aggiornare il nome e, se vuole, cambiare la password – con i requisiti di robustezza indicati sotto il campo. L’email, invece, non è modificabile, perché funge da identificativo per il login.

C’è anche la possibilità di eliminare il proprio account. Questa è una funzione delicata, e infatti, come vedremo, è soggetta a vincoli lato server: un docente non può essere cancellato se ha ancora materie o esami associati.

Dashboard Segreteria

Passiamo ora all’area della segreteria. La struttura è la stessa, ma con più poteri. Già dalla barra di navigazione in alto si vedono **più voci**: oltre a esami, sessioni, insegnamenti e corsi, compaiono “Docenti” e “Segretari”, che il docente non ha.

La dashboard della segreteria mostra indicatori d’insieme su tutto l’ateneo – sessioni totali, sessioni in corso, esami totali – e lo stesso calendario, ma con una vista globale: la segreteria vede la programmazione di tutti, non solo la propria.

Esami (Segreteria)

La lista esami della segreteria è simile a quella del docente, sempre raggruppata per sessione, ma con una differenza importante: c’è la **colonna del docente**, perché qui si vedono *gli appelli di tutti*.

E soprattutto la segreteria può **modificare ed eliminare qualsiasi esame, senza restrizioni**: non vale il limite degli esami già passati che ha invece il docente. È il ruolo amministrativo che si traduce in concreto.

Sessioni (Segreteria)

Le sessioni: stessa suddivisione in corso, pianificate e passate che vedeva il docente, ma qui la segreteria ha i **pulsanti di azione** – modifica ed elimina – e in alto il pulsante per creare una nuova sessione. La segreteria è infatti l’unica a poter definire quando si svolgono le sessioni e quando si aprono le finestre di pianificazione.

Nuova sessione (Segreteria)

Questo è il form di creazione di una sessione, e qui si vede bene quella doppia natura di cui ho parlato più volte. Ci sono due blocchi distinti: il **periodo della sessione** – inizio e fine della sessione vera e propria – e la **finestra di pianificazione degli appelli** – inizio e fine del periodo entro cui i docenti possono inserire gli esami.

Separare i due intervalli dà alla segreteria un controllo fine: per esempio può aprire le pianificazioni con largo anticipo rispetto al periodo d'esame, e chiuderle qualche giorno prima, per congelare il calendario.

Insegnamenti (Segreteria)

Per gli insegnamenti, la segreteria vede l'**elenco completo di tutto l'ateneo** – non solo i propri, perché non ne ha – con la possibilità di modificare ed eliminare, e il pulsante per crearne di nuovi. È qui che la segreteria associa una materia a un corso di laurea e a un docente titolare.

Nuovo insegnamento (Segreteria)

Il form di creazione di un insegnamento: nome, corso di laurea, docente titolare, anno e CFU. Da notare che il docente titolare si sceglie da un menu a tendina con i docenti esistenti: è il modo in cui la segreteria collega una materia a chi la insegna, abilitando poi quel docente a creare gli appelli relativi.

Corsi di laurea (Segreteria)

I corsi di laurea: la segreteria ne ha la gestione completa – crea, modifica, elimina – mentre il docente, come abbiamo visto, li vedeva solo in lettura. Per ogni corso: nome, durata in anni e dipartimento.

Nuovo corso di laurea (Segreteria)

Il relativo form di creazione: nome del corso, durata in anni e dipartimento scelto da una lista predefinita. Semplice e diretto.

Docenti (Segreteria)

Qui la segreteria gestisce gli account dei **docenti**: nome ed email, con la possibilità di modificarli, eliminarli e crearne di nuovi.

Vorrei chiarire un punto architetturale importante che sta dietro a questa pagina: la creazione di un docente *non* è una semplice registrazione. Quando la segreteria crea un docente, il sistema crea **in modo atomico** sia l'utente con le sue credenziali sia il relativo profilo di docente, con il ruolo forzato dal server. Se qualcosa va storto a metà, si fa il *rollback* di tutto, così non restano account incompleti.

Nuovo docente (Segreteria)

Il form di creazione di un nuovo docente: nome, email e password, con i requisiti di robustezza della password indicati sotto il campo. Dietro a questo semplice form, come ho appena detto,

c'è l'operazione atomica che crea utente e profilo insieme.

Segretari (Segreteria)

In modo speculare ai docenti, la segreteria gestisce anche gli **altri membri della segreteria**. Notate però un dettaglio sulla riga del mio account: accanto al mio nome c'è l'etichetta “Tu”, e i pulsanti di modifica ed elimina compaiono *solo sulla mia riga*. Sulle righe degli altri segretari non ci sono.

Questo è il principio del **self-only**: un membro della segreteria può modificare o eliminare solo il proprio profilo, non quello dei colleghi. È una scelta di sicurezza per evitare che gli amministratori si cancellino a vicenda.

Nuovo segretario

Il form per creare un nuovo segretario è identico, nella forma, a quello del docente – nome, email, password – ma il ruolo assegnato è quello di segreteria. Anche qui la creazione di utente e profilo è atomica.

Modifica segretario / Area personale

E questa è l'area personale del segretario, del tutto analoga a quella del docente: posso aggiornare nome e password, l'email resta fissa come identificativo, e posso eliminare il mio account, sempre nel rispetto della regola self-only.

Regole di dominio (parte 1)

Arriviamo a quella che, secondo me, è la parte più interessante del progetto: le **regole di dominio**. Sono i vincoli che rendono questo un vero sistema di pianificazione e non un semplice gestionale di dati. Sono tutte implementate e verificate lato server, indipendentemente da ciò che fa l'interfaccia.

La regola generale: la *segreteria gestisce tutte le risorse, il docente solo i propri dati*. Da qui discendono le altre. Il docente può creare esami *solo per le proprie materie*. Può modificare o eliminare *solo i propri esami*, e *solo se non sono già passati* – ecco perché prima vedevamo “Esame concluso” senza pulsanti. La segreteria, invece, può modificare o eliminare qualsiasi esame senza restrizioni. E gli esami devono cadere all'interno dell'intervallo di date della sessione di riferimento.

Regole di dominio (parte 2)

Continuano i vincoli, e qui ci sono quelli che secondo me danno valore al sistema.

Il docente può pianificare *solo durante la finestra di pianificazione* della sessione: fuori da quella finestra l'inserimento è bloccato. Non si possono pianificare esami di **sabato e domenica**. Non sono ammessi **conflitti**: per uno stesso corso di laurea, stesso anno e stesso giorno può esistere un solo esame – così gli studenti non si trovano due appelli sovrapposti. Non sono ammessi **duplicati**: una materia può avere un solo esame per sessione. L'ora di inizio deve precedere l'ora di fine. E infine, come già detto, la creazione di docenti e segretari avviene in modo atomico, con rollback in caso di errore.

Regole di dominio (parte 3)

Due ultime regole. La prima la conosciamo: la segreteria può modificare o eliminare *esclusivamente il proprio profilo* – il self-only. La seconda è un vincolo di integrità: un docente può essere eliminato *solo se non ha materie o esami associati*. Questo evita di lasciare nel sistema appelli “orfani”, senza un docente di riferimento.

Tutti questi controlli, ci tengo a ripeterlo, vivono nel service lato server: anche se qualcuno aggirasse l’interfaccia e chiamasse direttamente le API, i vincoli reggerebbero.

Progetto Coarse

Le ultime due slide riguardano la progettazione dell’ipertesto, cioè il metodo di design che abbiamo seguito durante il corso. Prima di scrivere il codice, ho progettato la navigazione.

Questo è il **progetto Coarse**, cioè la vista “a grana grossa”. In alto c’è l’area pubblica con il login, da cui si ottiene il token. Subito sotto l’area protetta, comune a entrambi i ruoli: dashboard, navbar, profilo e logout. Poi le due grandi aree separate – **Segreteria** e **Docenti** – ciascuna con i propri gruppi di pagine. Si vede a colpo d’occhio quanto l’area segreteria sia più ricca: ha la gestione di esami, sessioni, insegnamenti, corsi, docenti e segretari; il docente ha un sottoinsieme, centrato sui propri esami. Questo schema è la traduzione visiva dei permessi della matrice CRUD.

Progettazione dell’ipertesto

E questa è la vista di dettaglio, lo **schema di navigazione completo** in stile WebML. Ogni rettangolo è una pagina, ogni freccia un possibile percorso dell’utente.

Si riconoscono i due grandi blocchi, segreteria a sinistra e docente a destra, ciascuno con le proprie pagine di lista, di creazione e di modifica. In basso le unità operative – crea, aggiorna, elimina elemento – e una pagina di errore centralizzata, verso cui convergono i percorsi rossi quando qualcosa va storto. I percorsi verdi, invece, sono i ritorni dopo un’operazione andata a buon fine.

Il valore di questo diagramma è che il codice del front-end non l’ho improvvisato: è la realizzazione fedele di questo schema, progettato prima. Ogni pagina che vi ho mostrato ha una sua casella qui dentro.

Chiusura

Per concludere. Ho realizzato un sistema di pianificazione degli appelli con un’architettura full-stack a strati ben separati: un back-end NestJS con un pattern uniforme per ogni entità, una netta divisione tra dati, validazione, logica e autorizzazione, e un front-end React progettato a partire da uno schema di ipertesto.

Il filo conduttore di tutto il progetto sono i **vincoli di dominio**: finestre di pianificazione, niente weekend, niente conflitti né duplicati, permessi per ruolo, self-only, operazioni atomiche. Sono questi a trasformare una semplice applicazione CRUD in un vero strumento di pianificazione.

Vi ringrazio per l’attenzione. Sono a disposizione per le vostre domande.